Navigation Quiz — Answer Key

Detailed rationale, including why each incorrect option is wrong.

Projects: NavDetailList · NavThreeScreen · NavFourScreen · NavViewModelState · NavDataLayer

---

All six projects use **Navigation 3 (Nav3)**: a single Activity holds a back stack of `NavKey` objects, and `NavDisplay` swaps the screen whenever the top key changes. There is **no** `NavController`, `NavHost`, `navigate("route")`, or XML nav graph in any of these projects — which is why many distractors (drawn from the older Navigation Compose API) are wrong.

Blackboard randomizes option order on screen, so match answers by their text, not by position.

Q1. What renders the top of the back stack?

Which composable renders whichever key is currently on top of the back stack?

✓ CORRECT — NAVDISPLAY

`NavDisplay` reads the back stack, draws whichever key is on top, and animates the transition when the top changes. In every project: `NavDisplay(backStack = backStack, onBack = …, entryProvider = …)`.

WHY THE OTHERS ARE WRONG

- ✕ **NavHost** — the Navigation Compose (Nav2) host; not used here. It needs a `NavController` and string routes.
- ✕ **Scaffold** — a Material 3 layout frame supplying insets/`innerPadding`; it hosts the UI but knows nothing about the back stack.
- ✕ **NavController** — the Nav2 object that drives navigation; it does not exist in Nav3. The back stack (a list of keys) plays that role instead.
- ✕ **LazyColumn** — a scrolling list used inside a screen to render rows; unrelated to switching screens.

Q2. Creating and seeding the back stack

Which call creates the back stack, seeds it with the first key, and preserves it across recomposition and rotation?

✓ CORRECT — REMEMBERNAVBACKSTACK(CATEGORIESKEY)

This Nav3 function builds the back stack, seeds it with the start key, and **remembers** it across recomposition and configuration changes. Pushing a key navigates forward; popping navigates back.

WHY THE OTHERS ARE WRONG

- ✕ `remember { mutableStateOf(CategoriesKey) }` — holds a single key, not a stack, and is lost on rotation. It cannot represent `[Categories, Items, Detail]`.
- ✕ `rememberNavController()` — a Nav2 API; there is no `NavController` in Nav3, so it would not compile here.
- ✕ `rememberSaveable { CategoriesKey }` — saves a small Bundle-able value; it is not a back stack and offers no push/pop.
- ✕ `NavBackStack(CategoriesKey)` — calling it raw (without `remember…`) would rebuild a fresh stack every recomposition, losing all history.

Q3. `data object` key vs `data class` key

CategoriesKey is a `data object` but DetailKey is a `data class`. Why the difference?

✓ CORRECT

DetailKey needs an itemId argument; the list screen needs none.

A key names a destination *and* carries its arguments. `CategoriesKey` / `ListKey` is a singleton `data object` (one screen, no data). `DetailKey(val itemId: Int)` is a `data class` so each instance carries a different argument — `DetailKey(1)` vs `DetailKey(5)` land on the same entry block with different data.

WHY THE OTHERS ARE WRONG

- ✕ **"A data object cannot implement NavKey"** — false; `data object CategoriesKey : NavKey` does exactly that in the code.
- ✕ **"A data class loads measurably faster"** — no performance difference; the choice is about carrying arguments, not speed.
- ✕ **"The first screen must always be a data object"** — not a rule; a start destination could carry arguments and be a data class.
- ✕ **"Only a data class can be @Serializable"** — false; `@Serializable data object CategoriesKey` is valid and required.

Q4. What `backStack.add(DetailKey(id))` does

In NavDetailList, tapping a row runs `onOpen = { id -> backStack.add(DetailKey(id)) }`. What does this do?

✓ CORRECT

It adds a key to the back stack for NavDisplay to match.

"The jump." The tap does not name or call a screen — it appends a key. `NavDisplay` matches that key's **type** to the matching `entry<…> { }` block and runs it; the `id` only chooses which data the screen shows.

WHY THE OTHERS ARE WRONG

- ✕ **"It calls `navController.navigate("detail")`"** — that is the Nav2 string-route API, not used here; there is no `NavController`.
- ✕ **"It fires an Intent that starts a separate Activity"** — Nav3 is single-Activity; no Intents, no new Activities.
- ✕ **"It directly invokes the DetailScreen composable by name"** — screens are invoked only by `NavDisplay` via the matched entry block.
- ✕ **"It tears down NavDisplay and builds a replacement"** — `NavDisplay` stays mounted and re-renders the new top key; the previous key stays in the stack.

Q5. NavFourScreen "Start over"

`while (backStack.size > 1) backStack.removeLastOrNull()` — what is the result?

✓ CORRECT

All keys but the first pop, returning to the root.

The loop pops keys until only the bottom one (`CategoriesKey`) remains, jumping back to the first screen in one action — no matter how deep the user drilled (Categories → Items → Detail → Fact).

WHY THE OTHERS ARE WRONG

- ✕ **"The app exits because the back stack is emptied"** — the condition is `size > 1`, so it stops at one key; the stack is never emptied.
- ✕ **"Only the single top key pops"** — that describes a plain `removeLastOrNull()`; the `while` loop pops repeatedly.
- ✕ **"The back stack is duplicated, doubling every screen"** — `removeLastOrNull()` only removes; nothing adds or copies keys.
- ✕ **"It navigates forward and opens a fifth screen"** — removing keys is the opposite of navigating forward.

Q6. What `entry<DetailKey> { key -> DetailScreen(…) }` does

Inside `entryProvider`, what does this block do?

✓ CORRECT

It registers the screen for that key and passes the key in.

Like a `case` in a switch, `entry<KeyType> { }` **registers** which composable to show when that key type is on top. The body runs only when a matching key reaches the top, and the lambda receives the key instance so it can read its arguments (`key.itemId`).

WHY THE OTHERS ARE WRONG

- ✕ **"It renders DetailScreen immediately at launch"** — registration is not execution; it runs only when a `DetailKey` is on top, never at launch.
- ✕ **"It permanently removes every DetailKey from the stack"** — `entry` registers a destination; it never mutates the back stack.
- ✕ **"It converts the DetailKey data class into a data object"** — `entry` does no type conversion.
- ✕ **"It sets the back-button behavior for every screen"** — back behavior is set by `NavDisplay`'s `onBack = { backStack.removeLastOrNull() }`.

Q7. Why keys carry only an id, not the whole object

Why does a key carry `DetailKey(itemId)` instead of the whole Item?

✓ CORRECT

The id stays small and serializable; the screen looks it up.

Keys must be `@Serializable` (to survive process death). An `Int` id keeps the key tiny; the screen resolves the full object via a lookup — e.g. `itemById(key.itemId)` — keeping a single source of truth.

WHY THE OTHERS ARE WRONG

- ✕ **"Nav3 keys are forbidden from holding properties"** — false; `DetailKey(val itemId: Int)` holds one. The aim is to keep it small and serializable.
- ✕ **"Passing the whole object is faster but drains battery"** — not a battery issue; the concerns are serialization size and a single source of truth.
- ✕ **"The Item class cannot be referenced inside a composable"** — it is referenced constantly (e.g. `DetailScreen(item: Item)`).
- ✕ **"The id is the only field Item contains"** — Item has several fields (`title` , `blurb` , `categoryId` , sometimes `fact`).

Q8. Why ViewModel state survives rotation but `remember {}` does not

In NavViewModelState, why does the favorites set survive rotation while a `remember {}` value would not?

✓ CORRECT

It is scoped to the Activity and re-attached after recreation.

A `ViewModel` lives in a `ViewModelStore` owned by the Activity. On a configuration change Android hands the *same* store to the recreated Activity, so `viewModel()` returns the same `FavoritesViewModel` with its data intact. A `remember {}` value lives only in the composition, which rotation discards.

WHY THE OTHERS ARE WRONG

- ✕ **"`remember` clears its value every few seconds"** — it has no timer; it is lost only when the composition leaves (e.g. rotation).
- ✕ **"The ViewModel writes each value to a remote server"** — no network involved; survival comes purely from scoping. (Process death would need `SavedStateHandle`.)
- ✕ **"Rotation never recreates the Activity"** — it does by default; that recreation is the problem the ViewModel solves.
- ✕ **"A ViewModel keeps its data inside the nav key"** — it does not; favorites live in the ViewModel, keys carry only ids.

Q9. How the Detail screen changes a favorite (UDF)

In NavViewModelState's unidirectional data flow, how does the Detail screen change a favorite?

✓ CORRECT

It calls `viewModel.toggleFavorite(id)` ; state flows back down.

UDF: events flow up, state flows down. The button calls `toggleFavorite(item.id)` ; the ViewModel updates its private `MutableStateFlow` ; the new value emits through the read-only `StateFlow` , observed via `collectAsStateWithLifecycle()` , recomposing the screens. The composable never owns or mutates the data.

WHY THE OTHERS ARE WRONG

- ✕ **"The composable mutates the favorites Set directly"** — breaks UDF and is exactly what the project forbids; `_favorites` is private to the ViewModel.
- ✕ **"It writes the change into a `rememberSaveable` variable"** — that is used only for a local contrast counter; it is per-composable and could not be shared with the Items list.
- ✕ **"It edits the static `sampleItems` list"** — that is fixed sample data with no favorite flag; favorites are a separate `Set<Int>` of ids.
- ✕ **"It pushes a new key onto the back stack"** — toggling is a state change, not navigation; you stay on the Detail screen.

Q10. NavDataLayer's sealed `PlanetsUiState`

The list screen renders from a sealed `PlanetsUiState` . Which set of states does it model?

✓ CORRECT

Loading, Empty, Error, and Success.

A `sealed interface` with exactly these four subtypes makes the screen's `when (uiState)` **exhaustive** — the compiler forces handling of every case. `Loading` shows a spinner, `Empty` a note, `Error(message)` a message + Retry, `Success(planets)` the list.

WHY THE OTHERS ARE WRONG

- ✕ **"Start, Running, Paused, Stopped"** — a generic state machine / thread states, not this UI model.
- ✕ **"Foreground, Background, Created, Destroyed"** — Android lifecycle terms, a different concept.
- ✕ **"Push, Pop, Peek, Clear"** — stack operations (relevant to the back stack), not UI states.
- ✕ **"Idle, Tap, Drag, Release"** — gesture/input phases, unrelated to data-loading state.

Quick answer summary

#	CORRECT ANSWER	PROJECT
1	NavDisplay	all
2	`rememberNavBackStack(CategoriesKey)`	all
3	DetailKey needs an itemId argument; the list screen needs none	all
4	It adds a key to the back stack for NavDisplay to match	NavDetailList
5	All keys but the first pop, returning to the root	NavFourScreen
6	It registers the screen for that key and passes the key in	all
7	The id stays small and serializable; the screen looks it up	all
8	It is scoped to the Activity and re-attached after recreation	NavViewModelState
9	It calls `viewModel.toggleFavorite(id)` ; state flows back down	NavViewModelState
10	Loading, Empty, Error, and Success	NavDataLayer